

Music API

Full-Stack Software Engineering & Content Aggregation | API-Driven Application Architecture

Author: Keeno Smith

Project Type: Full-Stack Software Engineering Project

Platform: Web-based Single-Page Application (SPA)

Implementation: React / Vite frontend + Node.js / Express REST API + MongoDB / Mongoose persistence + Deezer and iTunes Podcast integrations

Primary Objective: Engineer a full-stack music application that combines external music and podcast data with persistent user-library functionality, music discovery, search, content aggregation, playlist data, caching, and browser-based audio playback.

Technical Executive Summary

The Music API is a full-stack JavaScript application engineered around a React single-page frontend, a Node.js/Express REST backend, MongoDB persistence through Mongoose, and external content-provider integrations.

The system provides a unified application layer over external music and podcast services rather than exposing third-party APIs directly to the frontend. Deezer is used as the primary music provider for tracks, albums, artists, categories, search, related artists, and music previews, while Apple iTunes Search and podcast RSS feeds provide podcast discovery and episode data.

The architecture separates frontend presentation from backend application logic and external provider communication. React pages and shared components communicate with frontend service modules, which consume backend REST endpoints. The backend then delegates provider-specific operations to application services and provider adapters. This creates a layered flow:

React UI → Frontend Service → Express Route → Controller → Application Service → External Provider / MongoDB → Normalized Response → React State

The application also maintains persistent domain data locally. MongoDB stores users, tracks, playlists, favourites, pinned songs, library entries, and related metadata. External music records are normalized into a local **Track** model using the external Deezer identifier as a stable provider key.

A centralized React application state architecture manages cross-page playback. The application maintains the active track, queue, current index, playback position, volume, shuffle/repeat state, autoplay state, and audio element reference at the application layout level. Both the compact floating player and expanded player operate against the same playback state rather than maintaining separate playback implementations.

The application implements multiple caching strategies. The backend uses an in-memory TTL cache for selected generated content, while the frontend uses both React-level cache state and browser **localStorage** for frequently accessed content. This reduces repeated external provider requests and allows previously retrieved content to be reused during a session.

The project demonstrates practical full-stack engineering capability across frontend architecture, REST API development, external API integration, data transformation, MongoDB modelling, asynchronous programming, state management, browser audio APIs, caching, CRUD operations, reusable component architecture, and development tooling.

The current implementation is a **local development application rather than a production-deployed system**. Its architecture demonstrates substantial application engineering, while several production concerns, including formal authentication, multi-user isolation, environment-based API configuration, automated testing, centralized error handling, and deployment infrastructure, remain opportunities for further hardening

Sector / Domain

Sector: Software Engineering / Digital Media / Music Technology

Domain: Music discovery, digital media aggregation, personal music libraries, podcast discovery, and browser-based audio playback.

The project models a digital media application where external content providers supply the primary music and podcast catalogue while the application provides its own discovery, organization, persistence, and playback layer.

What This Project Does

The application provides a unified interface for discovering and interacting with music and podcast content.

Core capabilities include:

- Music search
- Artist discovery
- Album discovery
- Category and genre-based discovery
- Curated music sections
- New music content
- Radio-style artist discovery
- Podcast discovery
- Podcast episode retrieval
- Album-level playback
- Artist-based playback
- Category-based playback
- Music preview playback
- Playback queues
- Queue reordering
- Shuffle
- Repeat-one playback
- Automatic queue extension
- Related-artist discovery
- Personal music library
- Favourite tracks
- Pinned tracks
- Recently added library content
- Playlist data
- Light/dark theme switching
- Persistent client-side content caching
- Persistent MongoDB user data
- Contextual actions through reusable menus
- Floating and expanded playback interfaces

Rather than storing the complete external music catalogue locally, the application retrieves catalogue information dynamically from external providers and persists selected application-level resources locally.

This creates a hybrid architecture combining:

External content discovery + application-owned state + persistent user data + local playback orchestration.

Key Role

Full-Stack Software Engineer

The project demonstrates responsibility across the application lifecycle rather than isolated frontend or backend implementation.

Responsibilities included:

- Designing the frontend application architecture
 - Designing route and page composition
 - Implementing reusable React components
 - Implementing centralized application state
 - Designing the browser playback architecture
 - Building frontend API service abstractions
 - Designing REST endpoints
 - Implementing Express routes and controllers
 - Building backend service layers
 - Integrating external APIs
 - Transforming third-party responses into application models
 - Designing MongoDB/Mongoose schemas
 - Implementing persistence workflows
 - Implementing content caching
 - Implementing user-library operations
 - Handling asynchronous data flows
 - Managing loading and error states
 - Configuring local development tooling
 - Debugging cross-layer application behaviour
 - Identifying production-hardening requirements
-

Responsibilities

Frontend

- React application composition
- React Router route architecture
- Context-based application state
- Theme management
- Playback state management
- Queue orchestration
- API integration
- Search orchestration
- Library interfaces
- User interaction flows
- Reusable glass UI primitives
- Browser audio integration
- Responsive application layout
- Loading and empty states

Backend

- Express application configuration
- REST route architecture
- Controller implementation
- Service-layer abstraction

- External provider integration
- Response normalization
- MongoDB persistence
- Mongoose data modelling
- User-library CRUD operations
- Content caching
- Error responses
- Seed data initialization

Integration

- Deezer API integration
 - iTunes Podcast Search integration
 - Podcast RSS parsing
 - External identifier normalization
 - Provider-specific response transformation
 - Internal API boundary design
-

Non-Technical Project Overview

The application addresses the problem of combining music discovery and personal organization into a single web application.

A user can discover music through curated sections, search for artists, albums and songs, browse categories, explore podcasts, and initiate playback. Content discovered from external providers can then become part of the application's own user-oriented state, including favourites, pinned songs and a personal library.

The project therefore moves beyond a simple external-API demonstration. The engineering challenge is in creating an application layer that manages the relationship between:

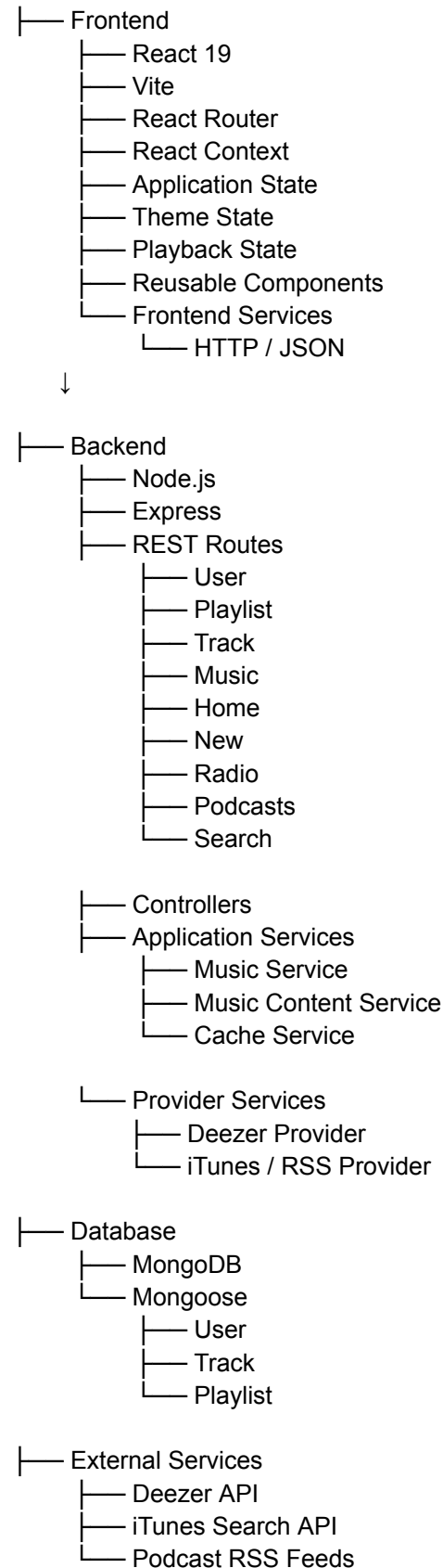
- externally sourced content,
- application-owned data,
- user interactions,
- persistent storage,
- playback state,
- caching,
- and multiple frontend presentation contexts.

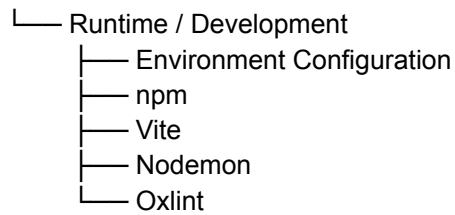
The result is a full-stack application architecture where external APIs provide catalogue information while the application controls the user experience, persistence model, playback behaviour and internal API contract.

System Architecture

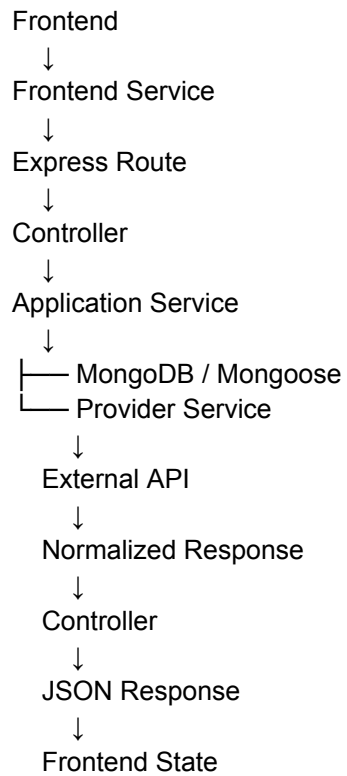
The application follows a layered full-stack architecture.

Music API

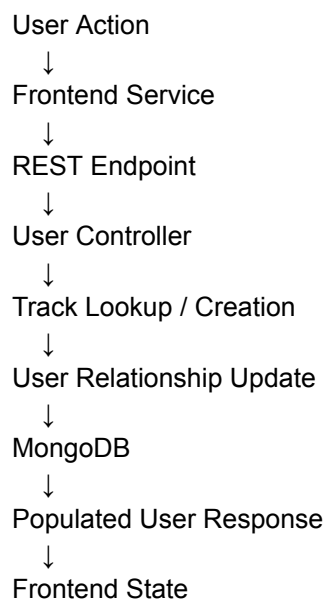




Primary Request / Data Flow



Persistence Flow



This represents the principal architectural boundary supported by the implementation.

The frontend does not directly integrate with every external provider. Instead, provider-specific logic resides on the backend, allowing the backend to transform provider responses into application-oriented structures.

Application Architecture

The frontend application is bootstrapped through `main.jsx`.

The application is wrapped in:

- React Strict Mode
- ThemeProvider
- BrowserRouter
- App

Routing is centralized through `AppRoutes`.

The application uses a shared `AppLayout` around the primary application routes. This layout provides:

- Sidebar
- Main route outlet
- Floating player
- Expanded player
- Application context
- Playback state
- User state
- Cache state
- Queue state
-

This makes `AppLayout` a significant application orchestration boundary rather than merely a visual wrapper.

The architecture effectively separates:

Application shell → **page routing** → **domain pages** → **reusable components** → **service layer** → **API**.

Frontend Application Architecture

Technology

The frontend uses:

- React 19.2.7
- React DOM 19.2.7
- React Router 7.18.1
- Vite 8.1.1
- Framer Motion 12.42.2
- Lucide React
- JavaScript / JSX
- Browser Fetch API
- Browser `localStorage`
- HTML5 Audio API

The project is JavaScript-based rather than TypeScript-based. The presence of React type packages in development dependencies does not change the implementation language.

Application Architecture

The frontend is organized around pages, shared components, layout components, contexts, and service modules.

The route architecture includes:

- `/`
- `/search`
- `/new`
- `/radio`
- `/podcasts`
- `/album/:id`
- `/artist/:id`
- `/podcast/:id`
- `/category/:id`
- `/recently-added`
- `/artists`
- `/albums`
- `/songs`
- `/playlists`
- `/favourites`

Dynamic route parameters allow resource-specific views to be resolved from identifiers.

Examples include:

```
/album/:id  
/artist/:id  
/podcast/:id  
/category/:id
```

This allows the same component architecture to operate against different external resources without creating separate hardcoded screens for each resource.

Theme Architecture

React Context is used for application-wide state transportation.

`AppContext` exposes the application state created by the main application layout.

The layout maintains state covering:

- Current user
- Authentication/sign-in flag
- Theme-related application state
- Content caches
- Queue visibility
- Lyrics visibility
- Playback state
- Current track
- Queue
- Original queue
- Current track index
- Autoplay queue
- Expanded player state
- Repeat state
- Shuffle state
- Volume
- Muting
- Previous volume
- Playback time
- Track duration

This allows independent pages and components to participate in the same playback lifecycle.

For example, an album page can establish the active queue while the `FloatingPlayer` consumes that queue without the album page owning the audio element itself.

Theme Architecture

Theme management is separated into its own context.

`ThemeProvider` maintains:

- Current mode
- Theme toggle operation
- Derived theme tokens

The token system combines:

- Colors
- Radius
- Spacing
- Blur
- Shadows
- Typography

The application therefore avoids hardcoding every theme property inside the theme provider itself.

Reusable components such as `GlassPanel`, `GlassMenu`, `GlassModal`, `ThemeToggle`, and `AppCanvas` consume the shared theme system.

This establishes a basic design-system architecture inside the application.

Search Architecture

Search is implemented as a multi-domain search operation.

The frontend waits 300 milliseconds after the search input changes before issuing a request. This provides client-side debouncing and avoids issuing a request for every keystroke.

The backend search operation executes:

- Track search
- Album search
- Artist search

in parallel through `Promise.all`.

The resulting response is normalized into:

```
{
  songs: [],
  albums: [],
  artists: []
}
```

This provides a single application-level search contract despite the underlying provider using separate search endpoints.

The backend also performs additional filtering after receiving Deezer results.

For songs, matching is based on:

- Song title
- Artist name

For albums:

- Album title
- Artist name

For artists:

- Artist name

This illustrates a provider-boundary approach where the application does not simply forward raw third-party responses.

Playback Architecture

Playback is one of the project's most substantial frontend engineering areas.

The audio lifecycle is centralized around a shared HTML Audio element.

The application maintains a single playback state across the floating and expanded player interfaces.

The playback system manages:

- Active track
- Active queue
- Original queue
- Current queue index
- Current playback position
- Track duration
- Playing/paused state
- Volume
- Muted state
- Repeat-one
- Shuffle
- Autoplay
- Queue visibility
- Expanded player state

The `FloatingPlayer` owns the audio element reference while the broader application state is controlled from the layout.

When the active track changes, the player updates the audio source from the track's `preview` property.

Playback events such as:

- `loadedmetadata`
- `timeupdate`
- `ended`

are used to synchronize the React state with the browser's audio state.

Queue Orchestration

The application implements client-side queue orchestration rather than treating each song as an isolated playback request.

Queue operations include:

- Play selected track
- Play next
- Add to queue
- Remove from queue
- Shuffle upcoming tracks
- Repeat current track
- Advance to next track
- Preserve played/current queue segment
- Dynamically extend the queue

The shuffle implementation preserves the current and previously played portion while randomizing upcoming tracks.

When a queue reaches its end, the player can request additional tracks through the backend autoplay endpoint.

This creates a continuous playback workflow:

Current Track → Queue Progression → Queue Exhaustion → Related/Curated Artist Resolution → Additional Tracks → Queue Extension

Autoplay / Related Artist Architecture

Autoplay is implemented as a backend orchestration process.

The system combines:

1. Curated artist lists
2. Deezer related-artist information
3. Artist popularity information
4. Deduplication
5. Track discovery
6. Randomization

The backend first determines whether the current artist belongs to one of the application's curated categories.

It then retrieves related artists from Deezer by:

- finding the requested artist,
- retrieving top tracks,
- inspecting contributors,
- retrieving artist details,
- ranking candidates by fan count.

The curated and provider-derived artist lists are combined and deduplicated.

The resulting artists are used to discover tracks, which are then randomized and deduplicated again.

This is a practical example of combining deterministic application content with external-provider discovery logic.

Content Aggregation Architecture

The backend contains `musicContentService.js`, which acts as a content-composition layer.

It builds structured datasets for:

- Home
- New
- Radio
- Podcasts
- Search

Rather than requiring the frontend to independently orchestrate numerous provider calls, the backend composes content into page-oriented response structures.

For example, Home is returned as sections containing:

- Section identifier
- Section title

- Content type
- Items

This allows the frontend to render the page from structured content rather than embedding provider-specific request logic into the page.

Backend Engineering

Technology:

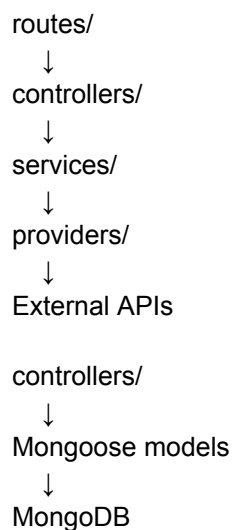
The backend uses:

- Node.js
- Express 5.2.1
- Mongoose 9.7.4
- MongoDB
- Axios
- CORS
- dotenv
- rss-parser
- Nodemon for development

The backend uses ES modules.

Backend Application Architecture

The backend follows a layered structure:



This separation provides a clear distinction between:

- HTTP endpoint definition
- HTTP request/response handling
- application orchestration
- external provider integration
- persistence

This is one of the strongest architectural characteristics of the project.

Express API Architecture

The backend exposes multiple REST-oriented route groups.

User API

GET /api/user
POST /api/user/favourite
POST /api/user/pin
POST /api/user/library
DELETE /api/user/library

Playlist API

GET /api/playlists

Track API

GET /api/tracks

Music API

GET /api/music/search
GET /api/music/album/:id
GET /api/music/artist/:name
GET /api/music/category/:category
GET /api/music/artist-info/:id
GET /api/music/podcast/:id
GET /api/music/category-artists/:category
GET /api/music/autoplay/:artist

Content APIs

GET /api/home
GET /api/new
GET /api/radio
GET /api/podcasts
GET /api/podcasts/play
GET /api/search

The API uses conventional HTTP methods for the available persistence operations, including **GET**, **POST**, and **DELETE**.

Controller Architecture

Controllers provide the HTTP boundary between Express and the application's services.

Examples include:

- `UserController`
- `playlistController`
- `trackController`
- `musicController`
- `homeController`
- `newController`
- `radioController`
- `podcastController`
- `searchController`

Controllers generally follow the pattern:

request → service/model operation → response

with `try/catch` handling around asynchronous operations.

This prevents route definitions from containing the majority of the business logic.

Service-Layer Architecture

The backend service layer provides an additional abstraction between controllers and external providers.

For music, `services/musicService.js` delegates to provider functions such as:

- `searchTracks`
- `getAlbumTracks`
- `getArtistTracks`
- `getCategoryTracks`
- `getArtistDetails`
- `getPodcastDetails`
- `getCategoryArtists`
- `getCategoryAlbums`
- `getAutoplay`

This creates an explicit provider boundary.

The controller does not need to know that a specific operation is implemented through Deezer or iTunes.

That separation makes the architecture more replaceable and easier to reason about.

External Provider Architecture

The provider layer currently contains:

```
services/providers/  
  deezerService.js  
  itunesPodcastService.js
```

Deezer

Deezer supplies:

- Tracks
- Albums
- Artists
- Artist metadata
- Related artists
- Categories
- Music previews
- Album track listings
- Search results

iTunes / Podcast RSS

iTunes supplies:

- Podcast search
- Podcast metadata
- Podcast feed URLs

RSS feeds supply:

- Podcast episodes
- Episode titles
- Episode media URLs
- Podcast artwork

The backend converts these provider responses into internal structures before returning them to the frontend.

Database Architecture

The persistence layer uses:

MongoDB + Mongoose

The principal schemas supplied are:

- `User`
- `Track`
- `Playlist`

Mongoose provides schema definitions, validation, timestamps, references, querying, population, and persistence.

Mongoose Relationships

The database uses references between domain models.

Examples:

User

```

├── favourites → Track[ ]
├── pinnedSongs → Track[ ]
└── library.track → Track
  
```

Playlist

```

└── tracks → Track[ ]
  
```

Controllers use Mongoose `populate()` to resolve these references before returning user or playlist data.

This means the frontend receives meaningful track objects rather than only MongoDB ObjectIds for these relationships.

User Library Workflow

The library demonstrates a complete persistence lifecycle.

When a user adds a song:

- The frontend sends the song payload to the user API.
- The backend derives the external track identifier.
- MongoDB is checked for an existing Track document.
- If none exists, a normalized Track is created.
- The user's library is inspected.
- Legacy library representations are normalized where necessary.
- The track is added with an `addedAt` timestamp.
- The user is saved.
- The updated user is populated.
- The updated user is returned to the frontend.
- React replaces its local user state with the returned representation.

This is a complete cross-layer workflow from UI action through persistence and state reconciliation.

Data Integrity and Compatibility

One notable implementation detail is compatibility handling within the user library.

The controller recognizes that library entries may exist in more than one representation:

```
{ track: ObjectId, addedAt: Date }
```

or legacy plain ObjectId entries.

The controller normalizes older entries before continuing.

This demonstrates awareness of data-shape evolution and backward compatibility rather than assuming that persisted data will always conform to the newest representation.

Caching Architecture

The application implements caching at two levels.

Backend Cache

`contentCache.js` uses a JavaScript `Map`.

Each cache entry stores:

```
data
expiresAt
```

The default TTL is:

10 minutes

The cache provides:

- `getCached`

- `setCached`
- `clearCached`
- `clearAllCache`

Expired entries are removed when accessed.

Backend caching is used for content such as:

- Home
- New
- Radio
- Podcasts
- Artist details
- Category artist lists

This is particularly relevant because these endpoints depend on external provider requests.

Frontend Cache

The frontend also maintains cache state for several content areas.

Examples include:

- Home cache
- New cache
- Radio cache
- Podcasts cache
- Album cache

The loading pattern is generally:

React Context Cache



localStorage Cache



Backend API



Persist into local cache

This reduces unnecessary network requests and provides faster subsequent access.

API Request and Response Handling

The frontend service layer consistently uses the Fetch API.

Requests generally:

- Build the endpoint URL
- Encode dynamic query/path parameters
- Perform the request
- Check `response.ok`
- Parse JSON

- Throw an explicit error when the request fails

For mutations, JSON request bodies are sent with:

Content-Type: **application/json**

The backend similarly returns JSON responses and explicit HTTP status codes for successful and failed operations.

Asynchronous Engineering

Asynchronous programming is used extensively throughout the system.

Examples include:

- React `useEffect` data loading
- Fetch requests
- Axios requests
- MongoDB queries
- Provider API requests
- RSS parsing
- Queue autoplay retrieval
- Parallel provider requests

The backend uses `Promise.all` where independent external requests can execute concurrently.

For example, artist details retrieves:

- Artist metadata
- Albums
- Top tracks

in parallel.

The search implementation similarly performs song, album and artist searches concurrently.

This reduces unnecessary sequential network latency.

Authentication & Security Boundaries

This area requires precise characterization.

The current application **does not implement a formal authentication system**.

The frontend uses:

`localStorage["signedIn"]`

as a sign-in state indicator.

The backend user controller retrieves the user through:

`User.findOne()`

rather than resolving a user from an authenticated request identity.

The current version has no:

- JWT authentication
- Session authentication
- Password hashing
- Login credentials
- Authentication middleware
- Authorization middleware
- Request-level user identity
- Role-based access control
- Per-user authorization checks

Therefore, the current user persistence architecture should be described as **single-user/global-user application state**, not secure multi-user authentication.

This distinction is important for technical accuracy.

User Scope

The application contains a user domain model and user-specific collections, including:

- Favourites
- Pinned songs
- Library
- Recently played
- Theme

However, the current backend retrieves the user globally with `User.findOne()`.

As implemented, this means the application does not yet provide true user isolation.

The data model is capable of representing user-specific state, but the request lifecycle does not yet establish which authenticated user is making the request.

This is an architectural boundary rather than simply a UI limitation.

Current Security Boundary

The present security model can therefore be characterized as:

Application-level user state without formal identity enforcement.

The backend does perform some validation, including:

- Required external track identifiers
- Missing-user handling
- Missing-track handling
- Schema-level validation
- External ID uniqueness

However, these controls do not replace authentication or authorization.

Error Handling

Error handling exists at multiple layers.

Frontend services check HTTP response status and throw explicit errors.

Backend controllers wrap asynchronous operations in `try/catch` blocks and return JSON error messages with HTTP 500 responses.

User operations also provide more specific statuses such as:

- `400` for missing track identifiers
- `404` when the user or requested track cannot be found
- `500` for unexpected server failures

This provides a basic but functional error boundary.

The current architecture does not yet include centralized Express error-handling middleware.

Development Configuration

The frontend service layer currently references:

`http://localhost:5050`

directly.

The backend uses:

`process.env.MONGO_URI`

for the MongoDB connection and:

`process.env.PORT || 5050`

for the server port.

The backend loads environment variables through `dotenv`.

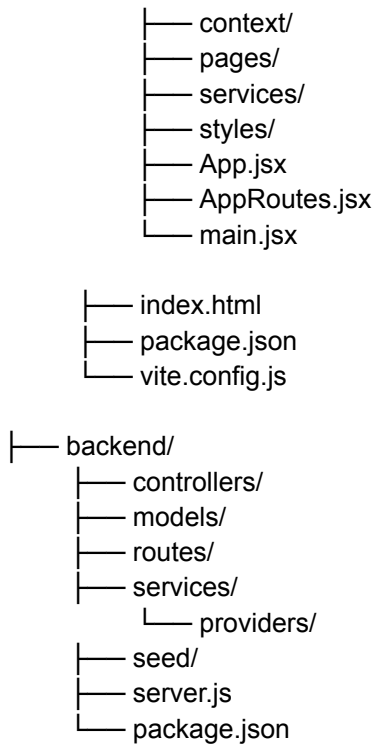
This establishes basic environment configuration on the backend, but the frontend API base URL is currently hardcoded and would need environment-based configuration for deployment.

Git Repository Structure

The project follows a conventional full-stack repository separation.

A representative structure is:

```
music-api/  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   │   ├── common/  
│   │   │   └── layout/
```



This reflects the supplied frontend/backend architecture. Exact root-level repository naming and any files not supplied should not be inferred beyond this structure.

Environment Structure

The backend expects an environment configuration containing at least:

MONGO_URI
PORT

PORT has a default of 5050.

The frontend currently does not demonstrate a Vite environment variable for the backend URL and instead uses the local backend address directly.

For a deployed environment, the frontend service layer should therefore be changed to consume an environment-specific API base URL.

Build & Development Tooling

Frontend

The frontend uses Vite.

Available scripts include:

```
npm run dev
npm run build
npm run lint
npm run preview
```

The project uses Oxlint for linting.

Backend

The backend provides:

```
npm start  
npm run dev  
npm run seed
```

Development uses Nodemon for automatic server restarts.

The seed command initializes the MongoDB database with development data.

Database Seeding

The project includes a database seed workflow.

The seed script:

- Connects to MongoDB
- Clears Users
- Clears Playlists
- Clears Tracks
- Creates a development user
- Inserts sample tracks
- Creates a playlist
- Assigns tracks to the user's library
- Assigns selected favourites
- Saves the user
- Disconnects from MongoDB

This provides a reproducible development dataset.

The seed operation is intentionally destructive because it clears the relevant collections before inserting seed data and should therefore be treated as a development initialization mechanism rather than a production migration strategy.

Deployment

Current Status: Not deployed.

The supplied implementation is configured for local development.

The backend defaults to: Port 5050

and the frontend service modules reference the local backend directly.

Problems Faced

Several engineering problems and implementation challenges are evident from the supplied code.

External API Data Complexity

The application depends on external provider response structures that differ from the application's internal domain structures.

The provider layer addresses this by explicitly mapping external responses into application-level objects.

Search Result Accuracy

Search required additional filtering beyond the raw Deezer search responses.

The implementation specifically filters results against the user's query for songs, albums and artists.

The supplied frontend also contains evidence of a known search-quality issue involving artist-like results such as "[The White Album](#)" being returned in an artist result context.

This indicates a broader challenge around semantic distinction between provider search results and the application's intended domain categories.

Queue Management

Playback is not simply a single-track operation.

The application has to coordinate:

- Current track
- Queue
- Queue index
- Shuffle
- Repeat
- Playback position
- Audio element state
- Autoplay
- Queue extension
- Multiple player interfaces

Centralizing this state at the application layout level addresses the problem of maintaining consistent playback across different pages and player views.

External API Dependency

Many content experiences depend on live external APIs.

The project therefore introduced caching at both backend and frontend levels to reduce unnecessary repeated provider calls.

Persistent Data Shape Evolution

The user library controller contains compatibility logic for both current structured library entries and older plain ObjectId representations.

This indicates the need to manage evolving persistence structures without immediately invalidating existing application data.

Provider Identity vs Local Identity

External tracks use Deezer identifiers while local MongoDB documents use ObjectIds.

The Track model bridges these two identity systems through `deezerId`.

This allows provider-derived content to become a persistent local domain resource

Engineering Decisions & Tradeoffs

Backend Provider Abstraction

Instead of embedding Deezer and iTunes calls directly into controllers, provider-specific logic is isolated in:
`services/providers/`

This reduces coupling between HTTP handling and external provider implementations.

Content Composition on the Backend

Home, New, Radio and Podcasts content is assembled on the backend.

This prevents the frontend from needing to understand the provider-specific orchestration required to construct each page.

The trade-off is that backend content services become responsible for curated content definitions

In-Memory Caching

The backend uses an in-process `Map` with TTL.

Advantages:

- Simple
- No additional infrastructure
- Low implementation complexity
- Appropriate for local development

Limitations:

- Cache is lost when the server restarts
- Cache is not shared between server instances
- It is not suitable as a distributed production cache

Frontend `localStorage` Caching

`localStorage` provides persistence across page reloads for selected content.

The trade-off is that stale content can remain available until replaced by newer data.

Centralized Playback State

Playback state is held at the layout/application level rather than separately in each page.

This increases the responsibility of **AppLayout**, but prevents multiple pages from creating independent playback states.

Engineering Scope

The project covers multiple engineering layers:

Layer	Scope
Presentation	React pages and reusable components
Application State	React Context and layout-level orchestration
Navigation	React Router
Client Integration	Fetch-based service layer
API	Express REST endpoints
Application Services	Content composition and provider abstraction
Provider Integration	Deezer, iTunes, RSS
Persistence	MongoDB / Mongoose
Domain Modelling	Users, tracks, playlists
Caching	Backend TTL + frontend localStorage
Playback	HTML Audio API
Development	Vite, Nodemon, Oxlint, npm
Configuration	dotenv, local API configuration

Entire Application Lifecycle & Workflow

The overall lifecycle can be represented as:

1. Application Initialization

```

Browser
↓
main.jsx
↓
ThemeProvider
↓
BrowserRouter
↓
App
↓
AppRoutes
  
```

↓
AppLayout

2. User Navigation

Route

↓
Page Component
↓
Frontend Service
↓
Backend REST Endpoint

3. Backend Processing

Express Route
↓
Controller
↓
Application Service
↓
Provider / MongoDB

4. Provider Retrieval

Application Service
↓
Deezer / iTunes / RSS
↓
Provider Response
↓
Normalization

5. API Response

Normalized Application Data
↓
Express JSON Response
↓
Frontend Service
↓
React State

6. Playback

User selects track
↓
Album / Artist / Category data retrieved
↓
Queue constructed
↓
Current track assigned
↓
HTML Audio source updated
↓
Audio playback

↓
timeupdate / ended events
↓
React playback state updated

7. Persistence

User action
↓
Frontend user service
↓
REST API
↓
User Controller
↓
Track lookup/create
↓
User relationship update
↓
MongoDB
↓
Populated User response
↓
React user state reconciliation

8. Caching

Request
↓
React Context Cache
↓
localStorage
↓
Backend
↓
Backend TTL Cache
↓
External Provider

This represents the principal application lifecycle supported by the supplied implementation.

Project Impact

The project's engineering value lies primarily in the breadth and integration of its technical layers.

It demonstrates the ability to build a system where multiple independent concerns operate together:

- external API integration,
- REST API design,
- persistent storage,
- domain modelling,
- frontend state,
- asynchronous workflows,

- caching,
- browser APIs,
- and reusable application architecture.

The project demonstrates that external APIs can be treated as infrastructure behind an application-specific service boundary rather than directly exposing provider implementation details to the UI.

The normalized Track model additionally demonstrates the ability to transform ephemeral external catalogue data into persistent application-owned resources.

The centralized playback architecture demonstrates the ability to manage state that spans multiple independent routes and UI surfaces.

From a portfolio perspective, the project therefore provides evidence of **full-stack system construction rather than isolated CRUD or frontend implementation**.

Technical Skills Demonstrated

Frontend Engineering

- React
- JSX
- React Context
- React Hooks
- React Router
- SPA architecture
- Component composition
- Shared application state
- Client-side caching
- localStorage
- Debounced search
- Async data loading
- Loading and empty states
- Browser Audio API
- Event listeners
- DOM interaction
- Responsive layout
- Reusable UI primitives
- Theme systems

Backend Engineering

- Node.js
- Express
- REST API design
- Routing
- Controllers
- Service-layer architecture
- Async/await
- Promise-based concurrency
- HTTP methods
- JSON APIs
- Error handling
- External service integration

Database Engineering

- MongoDB
- Mongoose
- Schema design
- ObjectId references
- `populate`
- CRUD operations
- Unique external identifiers
- Embedded relationship metadata
- Timestamped records
- Data-shape compatibility

Integration Engineering

- Deezer API
- iTunes Search API
- Podcast RSS feeds
- Axios
- RSS parsing
- External data normalization
- Provider abstraction

Application Engineering

- Queue orchestration
- Playback state management
- Cache architecture
- Content aggregation
- State reconciliation
- Data lifecycle management
- Development seed workflows

Tooling

- Vite
 - npm
 - Nodemon
 - Oxlint
 - dotenv
 - Git/GitHub-compatible project workflow
-

Technology Stack

Category Technologies

Frontend React 19.2.7, React DOM, JavaScript, JSX

Build Tool Vite 8.1.1

Routing React Router 7.18.1

UI / Icons Lucide React

Animation Framer Motion

State React Context + React Hooks

Browser APIs Fetch API, localStorage, HTML Audio API

Backend Node.js, Express 5.2.1

Database MongoDB

ODM Mongoose 9.7.4

HTTP Client Axios

External Music Provider Deezer

Podcast Provider iTunes Search

Podcast Data RSS feeds

RSS Parser rss-parser

Configuration dotenv

Development Nodemon

Linting Oxlint

Deployment Not deployed

What Could Be Improved

The following improvements are based directly on the current implementation rather than generic production checklists.

Formal Authentication

The largest architectural gap is the absence of actual authentication.

A production implementation should introduce:

- User registration
- Login
- Password hashing
- JWT or secure session authentication
- Authentication middleware
- Authenticated request identity
- Protected endpoints
- Logout/session invalidation

Multi-User Isolation

`User.findOne()` should be replaced with user identity derived from authenticated requests.

User-specific operations should become explicitly scoped:

Authenticated User
↓
User ID
↓
User-owned resource

This is essential for true multi-user deployment.

Frontend Environment Configuration

The frontend currently hardcodes:

`http://localhost:5050`

The API base URL should be moved into Vite environment configuration so development, staging and production environments can use different API endpoints without source-code changes.

Centralized Backend Error Handling

Instead of repeating `try/catch` blocks and error responses across controllers, the backend could introduce:

- Central error middleware
- Typed/application error classes
- Consistent error response format
- Request logging
- Provider-specific error translation

Automated Testing

No automated tests were supplied as part of the analysed implementation.

A production-oriented version should introduce:

Unit tests

For:

- Provider transformations
- Cache expiration
- Queue algorithms
- Search filtering
- Data normalization

Integration tests

For:

- REST endpoints
- MongoDB operations
- User-library workflows
- Favourite workflows

Frontend tests

For:

- Playback state
- Search behaviour
- Queue interactions
- User actions

Distributed Caching

The current **Map** cache is appropriate for a local application but would not scale across multiple backend instances.

A production architecture could use a shared cache such as Redis.

API Rate-Limit Protection

Because the system relies heavily on external APIs, production hardening should include:

- Request throttling
- Provider rate-limit handling
- Retry policies
- Backoff strategies
- Provider failure handling
- Circuit-breaking where appropriate

Provider Resilience

The application currently depends heavily on Deezer availability.

A more resilient architecture could introduce provider adapters with fallback behaviour, allowing alternative providers to be incorporated without changing the application layer.

Search Quality

The current search implementation combines provider search results with application-side filtering.

Search quality could be improved through:

- Exact-match prioritization
- Result relevance ranking
- Better artist/entity disambiguation
- Search result classification
- Pagination
- Result limits
- Cached search results

The known artist-result issue demonstrates why provider search semantics should not automatically be treated as application-level entity classification.

Remove Duplicate / Placeholder Logic

Several areas contain prototype remnants or incomplete functionality.

Examples include:

- Placeholder Albums page
- Artists page returning `null`
- Unwired playlist actions
- Unwired pin actions in some menus
- Unwired Suggest Less functionality
- Visual-only podcast shuffle
- Hardcoded content catalogues
- Hardcoded development user information
- Debug `console.log` statements
- Unused imports
- Duplicate album loading effect
- Unused component props
- Static placeholder values

These should be consolidated or removed before production release.

Production Hardening

A production-ready version would require a broader lifecycle around:

Development

↓

Automated Testing

↓

Environment Configuration

↓
Authentication
↓
Authorization
↓
Observability
↓
Build
↓
Deployment
↓
Monitoring
↓
Maintenance

The current project establishes the application architecture necessary to move toward this lifecycle but does not yet implement the full production delivery pipeline.

Architectural Strengths

The strongest architectural characteristics of the project are:

Layer separation: Routes, controllers, services and providers have distinct responsibilities.

Provider abstraction: External APIs are isolated behind backend provider services.

Data normalization: Third-party data is transformed into application-specific models.

Centralized playback: Playback state is shared across pages and player interfaces.

Persistent domain model: External tracks can become local application resources.

Cache hierarchy: Both backend and frontend caching strategies are present.

Reusable UI architecture: Shared menus, modals, panels and layout components reduce duplication.

Asynchronous orchestration: Independent provider operations are executed concurrently where appropriate.

Compatibility handling: The user controller accounts for legacy library data structures.

Architectural Limitations

The most significant limitations are:

- No formal authentication.
- No true multi-user authorization.
- Global `User.findOne()` persistence model.
- Hardcoded frontend API endpoint.
- No supplied automated test suite.

- No supplied CI/CD pipeline.
- No deployment infrastructure.
- In-memory backend cache.
- Several placeholder/incomplete frontend domains.
- Some duplicated and debug-oriented implementation logic.
- External provider dependency without advanced resilience mechanisms.
- Backend error handling remains controller-local rather than centralized.

These limitations should be understood as the current maturity boundary of the project rather than hidden weaknesses.

Overall Engineering Capability

The Music API demonstrates the ability to engineer a **multi-layer full-stack application around external data providers and application-owned state**.

The project goes beyond basic CRUD by requiring the coordination of:

- third-party APIs,
- provider-specific data transformation,
- REST API boundaries,
- MongoDB persistence,
- user-to-resource relationships,
- frontend state management,
- caching,
- asynchronous content aggregation,
- browser audio playback,
- dynamic queues,
- and reusable application infrastructure.

The implementation demonstrates particular strength in **integration architecture and state orchestration**.

The provider architecture shows an understanding of separating external systems from application logic. The Track model demonstrates how externally sourced entities can be normalized and persisted locally. The playback architecture demonstrates the ability to manage complex state across independent views. The caching implementation shows awareness of the cost and latency associated with repeated external requests.

The project also demonstrates an important engineering capability: recognizing the difference between an application that works functionally in a local environment and one that is ready for production.

The current system has a strong application foundation, while authentication, authorization, testing, deployment, observability, distributed caching and production configuration represent the next maturity stage.

Portfolio-Level Engineering Statement

The Music API demonstrates full-stack engineering capability through the design and implementation of a React-based SPA backed by a Node.js/Express REST API and MongoDB persistence layer, with external Deezer and iTunes/RSS integrations.

The project demonstrates the ability to build layered application architecture, design RESTful interfaces, normalize third-party data, model persistent relationships with Mongoose, implement user-scoped application functionality, coordinate centralized playback state, construct dynamic queues, introduce multi-level caching, and manage asynchronous data flows across frontend, backend and external services.

It provides evidence of practical experience building systems where **frontend state, backend services, persistent data, external APIs and browser capabilities must operate as one coherent application**.

Concise Portfolio Summary

Engineered a full-stack music discovery and playback platform using React, Vite, Node.js, Express, MongoDB and Mongoose, integrating Deezer for music catalogue data and iTunes/RSS for podcast content. Designed a layered REST architecture separating routes, controllers, application services and external provider adapters; implemented normalized track persistence, user libraries, favourites, pinned tracks and playlists; and developed centralized React playback orchestration supporting queues, shuffle, repeat, autoplay and shared floating/expanded player interfaces. Implemented backend TTL caching and frontend localStorage caching to reduce repeated external provider requests. The project demonstrates full-stack JavaScript engineering, API integration, data modelling, asynchronous orchestration, state management, browser audio integration and application architecture, with formal authentication, multi-user authorization, automated testing and production deployment identified as future hardening requirements.

Final Technical Classification

Project Classification: Full-Stack Application Engineering

Architectural Style: Layered SPA + REST API + External Provider Integration + Document Database

Primary Engineering Domains:

- Full-Stack Web Engineering
- React Application Architecture
- REST API Engineering
- MongoDB/Mongoose Data Modelling
- External API Integration
- Content Aggregation
- State Management
- Browser Audio Engineering
- Caching
- Asynchronous Application Design

Current Maturity: Substantial local full-stack implementation with production-hardening requirements

Deployment: Not deployed

Authentication: Not formally implemented

Persistence: MongoDB/Mongoose

Primary External Providers: Deezer, iTunes Search, Podcast RSS

Primary Engineering Demonstration: Designing and integrating a multi-layer application in which external content, persistent domain data, frontend state and browser playback functionality operate through defined application boundaries.
